

DevPilot 智能研发系统

可背诵版项目面试准备手册

目标：Java 后端 / 全栈工程师 / AI 应用工程师 / Agent 开发工程师

阅读口径：每个问题中的【参考回答】都可直接口述；【建议补充】给出面试中可采用的合理做法和说法，你应在面试前结合实际代码确认；【通用原理】用于支撑追问。已确认指标：180 条离线集，Context Recall 0.988、Hit Rate 0.988、MRR 0.935。

使用顺序：先背诵第一部分的一分钟稿；再把第二、五部分的建议补充逐项落实为真实代码和数据；最后按第三、四部分模拟追问。

第1部分：项目全景拆解

1. 一句话介绍

DevPilot 是面向金融研发场景的智能知识库与 Bugfix Agent：通过 Agentic RAG 为企业文档、错误码、配置和 SOP 提供带来源引用的回答，并基于日志与 Python 堆栈给出 Bug 定位、根因分析和修复建议。

2. 项目背景与业务痛点

【项目事实】企业文档、排障手册、错误码说明和代码相关资料需要被用于研发问答与排障。

【待补充】原有检索/排障方式、资料分散位置、人工耗时、典型高频故障，以及金融场景合规边界。

3. 目标用户与使用场景

【项目事实】研发人员；场景包括技术问答、问题排查、配置查询、SOP 咨询，以及根据异常日志和 Python 堆栈进行 Bug 定位。

【待补充】是否也面向运维/测试，权限隔离、多租户和敏感信息策略。

4. 输入、处理、输出

输入：企业文档/图片文档、用户问题、多轮历史、异常日志和 Python 堆栈。处理：解析/OCR、分块、向量化、混合检索、RRF、Reranker；意图路由、证据评估、改写重检；Plan-Execute-Replan。输出：带来源引用的回答，或候选代码位置、原因、影响范围和建议 Diff。

5. 技术架构与模块

接入层：FastAPI、SSE；工作流：LangGraph、LangChain、ReAct；检索层：解析/OCR、Embedding、Milvus、BM25、RRF、Reranker；状态层：PostgreSQL、LangGraph Checkpointer；代码分析：源码读取/搜索工具与 MCP。

【待补充】模型/Embedding/Reranker 型号和部署，BM25 存储实现、鉴权、监控、缓存和队列。

6. 数据流与请求链路

入库：文档解析或 RapidOCR → 清洗/分块 → Embedding → Milvus；并建立 BM25

所需倒排能力【待补充：具体实现】。问答：请求 → 意图路由 → Hybrid RAG → RRF → Reranker → 证据评估 → 必要时 Query 改写重检 → 证据约束回答 + 来源引用 → SSE。Bugfix：日志解析 → 堆栈定位 → 读取上下文 → 源码搜索 → 根因分析 → 修复建议。

7. 个人职责与贡献边界

【项目事实】你完成/基于五块能力：RAG 核心链路、Agentic RAG、离线评测、Bugfix Agent、会话与执行状态管理。

【待补充】每块的代码目录、PR/提交、亲自设计的接口/表/节点，以及同事负责的模型、前端、部署部分。不要将“系统具备”直接说成“我独立完成”。

8. 项目结果、业务与技术价值

【项目事实】180 条离线评测集上，Hybrid RAG: Context Recall 0.988、Hit Rate 0.988、MRR 0.935。技术价值是让研发问答具备可追溯证据，并辅助排障。

【待补充】实际用户、节省时间、采纳率、P95、成本、线上验收；离线指标不可替代业务收益。

9. 最有技术含量的 3 点

① 针对错误码、路径、配置项的 Dense + BM25 + RRF + Reranker。② “检索—证据评估—改写—重检”的 Agentic RAG 闭环。③ 基于持久化状态的 Plan-Execute-Replan Bugfix 流程。

10. 一分钟项目介绍稿

我参与开发 DevPilot，一个面向金融研发场景的智能知识库和 Bugfix Agent。我的核心工作是 RAG 和 Agent

链路：从文档解析、OCR、分块、向量化到 Milvus

检索和来源引用；针对错误码、日志路径、配置项等精确匹配问题，我实现 Dense 与 BM25 混合检索，结合 RRF 和 Reranker。在 180 条离线集上，Hybrid RAG 的 Context Recall 和 Hit Rate 都是 0.988，MRR 是 0.935。我还用 LangGraph 编排检索、证据评估、Query 改写和重检，并实现日志和 Python 堆栈驱动的 Bugfix 流程，支持重启后的会话和执行状态恢复。

三分钟项目介绍稿

项目解决金融研发资料分散、问答不可追溯、排障依赖人工经验的问题。资料经过解析/OCR、分块和 Embedding 后存入 Milvus；查询同时走稠密与 BM25 关键词检索，用 RRF 融合、Reranker

精排，再将证据与来源传给模型。这样兼顾语义问题和错误码、配置键、日志路径等字面实体。LangGraph

将意图路由、检索、证据充分性判断、改写重检和回答拆为节点。Bugfix Agent

则解析日志/堆栈，读取上下文、搜索关联源码，输出原因、影响范围和建议 Diff。PostgreSQL 与 Checkpointer

持久化会话摘要和执行状态。我会如实说明：现有材料验证的是离线检索指标，线上收益、性能与安全口径仍需补齐。

五分钟项目介绍稿

在三分钟稿基础上补充技术取舍：单纯向量检索对语义表达友好，但错误码、路径、配置项这类高辨识度 token

容易漏召回，因此采用 Hybrid；RRF 不要求两路分数同尺度，适合先融合候选，再用 Reranker 深度精排。Agentic RAG

的价值不是增加节点，而是在首次证据不足时控制模型改写查询并重检，最终以证据与引用约束回答。评测要讲清 180

条来源、标注规则、切分和基线；现有三指标主要证明检索排序，不证明生成绝对正确。Bugfix Agent 只承诺 Python

异常堆栈定位和建议 Diff，不等同自动修复。

第2部分：项目风险与真实性审查

“完成 RAG 核心链路”〔高〕

面试官会问：每层你写了什么？BM25 放在哪里？入库失败如何处理？

需要准备：接口、索引/表、异常处理、提交记录。

推荐真实表达：我负责 X 到 Y（以代码为准）；部署/模型服务由 X 负责。

指标 0.988 / 0.935 〔高〕

会问：Hit Rate 的 top-k？Context Recall 如何标？基线？

准备：公式、k、样本来源、标注规则、评测脚本、Dense-only/BM25-only/无 Reranker 对照。

表达：结果仅对应 180 条离线集和明确口径，不能代表线上满意度。

“增强精确检索能力”〔高〕

会问：给一个优化前失败、优化后成功的真实例子。

准备：至少 3 个错误码/路径/配置项案例及候选排序。

表达：用具体样例和同口径对比，避免“显著提升”但没有基线。

证据充分性评估〔高〕

会问：谁评估、prompt/阈值、最大重试、如何防循环？

准备：节点状态、退出条件、失败 fallback。

表达：材料只确认有评估和改写重检；配置细节以实现为准。

Bugfix 根因与建议 Diff 〔高〕

会问：如何防幻觉？能跨文件吗？Diff 执行过吗？

准备：工具白名单、堆栈规则、搜索范围、人工审核、成功/失败案例。

表达：是辅助分析和建议 Diff，不宣称自动修复或保证根因正确。

恢复与续执行〔高〕

会问：checkpoint 保存了什么？工具调用中断如何幂等？

准备：thread/run/step 标识、状态 schema、恢复测试。

表达：已知持久化会话记忆、摘要和执行状态；精确语义需确认。

金融数据安全〔高〕

会问：ACL、脱敏、审计、模型是否出域？

准备：实际鉴权、文档权限、日志脱敏、审计与保留策略。

表达：材料未提供安全实现，不能虚构；这是生产化必须补齐项。

技术选型〔中〕

会问：为什么 Milvus/LangGraph/PG，而不是 ES/自研/Redis？

准备：当时约束、数据量、已有基础设施、替代方案。

表达：选择必须绑定需求和约束，不说“行业常用”。

SSE、多轮摘要、异常容错〔中〕

会问：断连恢复、取消、摘要触发、重试/熔断？

准备：事件协议、超时、摘要策略、真实实现。

表达：材料确认能力存在，不虚构断点续传或具体重试次数。

性能、成本、线上价值缺失〔高〕

会问：QPS、P95、CPU、模型费用、用户收益？

准备：压测报告、trace、调用 token、资源规格、线上反馈。

表达：没有线上口径不报虚数，说明将如何压测和补齐。

第3部分：完整面试问题库与参考回答

问题 1：为什么做 DevPilot，成功如何定义？

背景/价值 | 中等 | 目的：判断是否理解业务。

关联：研发知识问答与排障。

【参考回答】【项目事实】系统为研发资料问答和 Bug 排查提供带引用答案/建议。成功分两层：离线检索找到正确证据、线上降低排障成本或提升采纳率。现有材料只证明前者：180 条上的 0.988/0.988/0.935；后者【建议补充】。

回答结构：痛点→方案→已验证→边界。关键数据：180

条与三项指标。追问：原有流程多慢？常见错误：把离线指标说成业务收益。复习：样本、基线、线上数据。

【回答方向建议】先用一句结论回答，再按“背景或问题 - 方案 - 取舍 - 验证或边界”展开；若被追问到具体参数、表结构或线上数据，以实际实现为准，不要临场虚构。

问题 2：你的个人贡献如何证明？

个人贡献 | 高难 | 目的：区分团队与个人。

关联：五项职责。

【参考回答】【项目事实】我负责 RAG 核心链路、Agentic RAG、评测、Bugfix 与状态管理。应按自己写的接口、检索融合逻辑、图节点和评测脚本举证，并说明协作边界。【建议补充】PR/commit、代码目录和团队分工。

结构：模块边界→决策→可验证产物→协作。追问：谁做部署/模型？错误：说“都是我做的”。

【回答方向建议】先用一句结论回答，再按“背景或问题 - 方案 - 取舍 - 验证或边界”展开；若被追问到具体参数、表结构或线上数据，以实际实现为准，不要临场虚构。

问题 3：完整问答请求链路是什么？

架构 | 中等 | 目的：端到端理解。

关联：LangGraph + Hybrid RAG + SSE。

【参考回答】FastAPI 接请求→意图路由→Dense/BM25 候选召回→RRF→Reranker→LLM

证据充分性判断→不足时改写重检→基于证据的带引用回答→SSE 返回；PG/Checkpointner

持久化会话和执行状态。【建议补充】鉴权、超时、各节点 schema。

追问：任一节点失败怎么办？错误：遗漏重检与状态。

【回答方向建议】先用一句结论回答，再按“背景或问题 - 方案 - 取舍 - 验证或边界”展开；若被追问到具体参数、表结构或线上数据，以实际实现为准，不要临场虚构。

问题 4：为什么用 Dense + BM25，而不是只用向量检索？

检索/选型 | 高难 | 目的：检索原理。

关联：错误码、日志路径、配置项。

【参考回答】【通用原理】Dense 擅长语义相似，BM25 对稀有且字面重要 token

更敏感；两路互补。【项目事实】系统用 Hybrid 增强精确实体检索。【建议补充】两路 top-k、BM25 实现、失败样例。

结构：对象特征→两路优缺点→融合→真实验证。追问：BM25 参数？错误：只说“向量不准”。

【回答方向建议】先用一句结论回答，再按“背景或问题 - 方案 - 取舍 - 验证或边界”展开；若被追问到具体参数、表结构或线上数据，以实际实现为准，不要临场虚构。

问题 5：RRF 为什么适合？公式是什么？

检索/算法 | 高难 | 目的：验证融合细节。

关联：项目使用 RRF。

【参考回答】典型公式 $score(d) = \sum 1/(k + rank_i(d))$ ，按排名融合，不依赖向量分和 BM25 分同尺度；先融合再由 Reranker 精排。【建议补充】实际 k、候选去重、top-k。

追问：为何不用线性加权？错误：混淆 RRF 的 k 与 top-k。

【回答方向建议】先用一句结论回答，再按“背景或问题 - 方案 - 取舍 - 验证或边界”展开；若被追问到具体参数、表结构或线上数据，以实际实现为准，不要临场虚构。

问题 6: Reranker 放在哪里，代价是什么？

检索/性能 | 中等 | 目的：效果-性能取舍。

关联：项目有精排。

【参考回答】先高召回拿有限候选，再对 query-document 对精排，避免全库交叉编码。额外代价是推理延迟和成本；可通过候选截断、批处理、缓存/路由控制。【建议补充】模型、候选数、P95。

错误：认为候选越多越好。

【回答方向建议】先用一句结论回答，再按“背景或问题 - 方案 - 取舍 - 验证或边界”展开；若被追问到具体参数、表结构或线上数据，以实际实现为准，不要临场虚构。

问题 7: 180 条评测集如何构建，如何避免泄漏？

评测 | 高难 | 目的：指标可信度。

关联：五类场景的评测集。

【参考回答】【项目事实】覆盖错误码、配置、日志路径、排障、SOP。【建议补充】来源、标注、gold chunk、切分。正确做法是开发调参与最终测试集隔离；同一原文的近重复 chunk 不跨集合；记录文档版本/时间。

追问：谁标注？错误：认为 RAGAS 自动给分即可。

【回答方向建议】先用一句结论回答，再按“背景或问题 - 方案 - 取舍 - 验证或边界”展开；若被追问到具体参数、表结构或线上数据，以实际实现为准，不要临场虚构。

问题 8: Context Recall、Hit Rate、MRR 分别是什么？

评测 | 高难 | 目的：避免误解指标。

【参考回答】Context Recall 衡量相关证据覆盖；Hit Rate 衡量 top-k 是否至少有一个相关结果；MRR 衡量第一个相关结果排名。具体定义、top-k 和是否人工/RAGAS

判定必须以评测脚本为准。它们不能证明最终答案无幻觉。

关键数据：0.988/0.988/0.935。

【回答方向建议】先用一句结论回答，再按“背景或问题 - 方案 - 取舍 - 验证或边界”展开；若被追问到具体参数、表结构或线上数据，以实际实现为准，不要临场虚构。

问题 9: 证据评估如何防止无限重检？

Agent | 高难 | 目的：状态机可靠性。

【参考回答】【项目事实】有评估、改写重检。应记录尝试次数/查询历史，设最大重试；到上限仍不足则明确“知识库证据不足”而非编造。【建议补充】真实阈值、模型与 fallback。

错误：说模型会自己停止。

【回答方向建议】先用一句结论回答，再按“背景或问题 - 方案 - 取舍 - 验证或边界”展开；若被追问到具体参数、表结构或线上数据，以实际实现为准，不要临场虚构。

问题 10: 强制知识库模式的意义？

产品/可信 AI | 中等 | 目的：幻觉控制。

【参考回答】要求回答受检索证据约束，适合需要可追溯的企业知识场景；无命中时应拒答或追问。引用降低风险但不保证绝对正确。【建议补充】无答案策略、引用校验。

【回答方向建议】先用一句结论回答，再按“背景或问题 - 方案 - 取舍 - 验证或边界”展开；若被追问到具体参数、表结构或线上数据，以实际实现为准，不要临场虚构。

问题 11: Bugfix Agent 的流程与边界？

Agent/代码分析 | 高难 | 目的：工程闭环。

【参考回答】【项目事实】日志解析→堆栈定位→上下文读取→源码搜索→根因分析→修复建议，Plan-Execute-Replan 让工具观察能修订计划。边界：支持 Python 堆栈；输出建议 Diff 供人工审核，不是自动提交/修复。

追问：跨文件、单测、权限？

【回答方向建议】先用一句结论回答，再按“背景或问题 - 方案 - 取舍 -

验证或边界”展开；若被追问到具体参数、表结构或线上数据，以实际实现为准，不要临场虚构。

问题 12：如何从 Python 堆栈定位可疑代码？

代码分析 | 高难 | 目的：实现深度。

【参考回答】一般解析 `traceback` 的文件路径、行号、函数、异常类型，过滤第三方帧或优先业务代码帧，读附近上下文并按错误信息搜索调用点。【建议补充】真实帧过滤、路径映射、异步异常规则。不要说最后一行必然是根因。

【回答方向建议】先用一句结论回答，再按“背景或问题 - 方案 - 取舍 - 验证或边界”展开；若被追问到具体参数、表结构或线上数据，以实际实现为准，不要临场虚构。

问题 13：建议 Diff 如何防止有害修改？

安全/质量 | 高难 | 目的：生成式代码风险。

【参考回答】【项目事实】定位为建议

Diff。应只读代码工具、路径白名单、输出证据/假设、人工审核；若没有语法/单测验证，明确还未覆盖。

追问：可否改生产配置？

【回答方向建议】先用一句结论回答，再按“背景或问题 - 方案 - 取舍 - 验证或边界”展开；若被追问到具体参数、表结构或线上数据，以实际实现为准，不要临场虚构。

问题 14：为什么用 LangGraph，如何恢复状态？

工作流/选型 | 高难 | 目的：框架与状态理解。

【参考回答】【项目事实】LangGraph 编排两类流程，Checkpoint + PG 持久化会话、摘要、执行状态并支持恢复。理由应落到有状态图、条件分支、中断恢复、可追踪节点。【建议补充】`thread_id/run_id`、`schema`、版本迁移。

【回答方向建议】先用一句结论回答，再按“背景或问题 - 方案 - 取舍 - 验证或边界”展开；若被追问到具体参数、表结构或线上数据，以实际实现为准，不要临场虚构。

问题 15：多轮摘要压缩丢事实怎么办？

会话/LLM | 中等 | 目的：上下文管理。

【参考回答】摘要控制上下文和成本但会丢信息。保留最近原文，将长期内容摘要化，关键约束结构化保留；冲突时以原始消息/检索证据优先。【建议补充】触发阈值与摘要策略。

【回答方向建议】先用一句结论回答，再按“背景或问题 - 方案 - 取舍 - 验证或边界”展开；若被追问到具体参数、表结构或线上数据，以实际实现为准，不要临场虚构。

问题 16：SSE 流式返回如何处理断连和错误？

接口/稳定性 | 中等 | 目的：工程可靠性。

【参考回答】定义 `event` 类型、`request/run id`、`error/done` 事件；可用 `Last-Event-ID/run id` 支撑恢复，服务端处理取消、代理超时和心跳。【建议补充】本项目是否真的实现断点续传。错误：说 SSE 天然可恢复。

【回答方向建议】先用一句结论回答，再按“背景或问题 - 方案 - 取舍 - 验证或边界”展开；若被追问到具体参数、表结构或线上数据，以实际实现为准，不要临场虚构。

问题 17：模型、OCR、检索失败如何降级？

稳定性 | 高难 | 目的：容错设计。

【参考回答】【项目事实】有异常容错但细节未给。应按节点做超时、有限重试与可见错误；OCR 失败可记录待处理；单路检索失败才在确认后降级；状态持久化支持恢复。【建议补充】重试/熔断/告警参数。

【回答方向建议】先用一句结论回答，再按“背景或问题 - 方案 - 取舍 - 验证或边界”展开；若被追问到具体参数、表结构或线上数据，以实际实现为准，不要临场虚构。

问题 18：Milvus 为什么适合，何时不适合？

存储/选型 | 中等 | 目的：选型边界。

【参考回答】Milvus 适合作为 `Embedding` 向量检索层；复杂关键词、权限和元数据仍需倒排/过滤层。小规模或已有 `ES/OpenSearch` 向量能力时，新增 Milvus 运维成本不一定值得。【建议补充】数据量、维度、索引、过滤。

【回答方向建议】先用一句结论回答，再按“背景或问题 - 方案 - 取舍 - 验证或边界”展开；若被追问到具体参数、表结构或线上数据，以实际实现为准，不要临场虚构。

问题 19：文档解析、OCR、分块如何保证质量？

数据/RAG | 高难 | 目的：上游数据质量。

【参考回答】【项目事实】有解析、RapidOCR、分块。chunk 应保留标题、页码、文档 ID、来源位置，避免切断错误码、表格、代码块。【建议补充】格式、长度/overlap、清洗、OCR 置信度和失败重试。

【回答方向建议】先用一句结论回答，再按“背景或问题 - 方案 - 取舍 - 验证或边界”展开；若被追问到具体参数、表结构或线上数据，以实际实现为准，不要临场虚构。

问题 20：来源引用如何可追溯？

可信度 | 中等 | 目的：证据链。

【参考回答】入库 chunk 应有稳定文档 ID、标题/路径、页码/段落和版本；检索元数据透传生成层，引用来自实际证据。【建议补充】引用格式、版本策略、claim-citation 校验。

【回答方向建议】先用一句结论回答，再按“背景或问题 - 方案 - 取舍 - 验证或边界”展开；若被追问到具体参数、表结构或线上数据，以实际实现为准，不要临场虚构。

问题 21：流量增长 10 倍如何扩容？

系统设计 | 高难 | 目的：容量意识。

【参考回答】先通过 trace/压测确认瓶颈；无状态 API 横向扩，耗时 OCR/入库与交互隔离，向量库按索引/分片扩，PG 做连接池/索引/归档，LLM 做限流、排队和成本控制。【建议补充】当前拓扑和容量。

【回答方向建议】先用一句结论回答，再按“背景或问题 - 方案 - 取舍 - 验证或边界”展开；若被追问到具体参数、表结构或线上数据，以实际实现为准，不要临场虚构。

问题 22：如何测试 RAG 与 Agent？

测试 | 高难 | 目的：非确定性系统验证。

【参考回答】检索层用固定 query/gold evidence
评三项指标：图节点对路由、退出、工具参数做单测/集成；生成层评事实性、引用和拒答；Bugfix 用已知异常回归。固定模型/参数、记录 prompt 与检索快照。【建议补充】实际 CI/覆盖率。

【回答方向建议】先用一句结论回答，再按“背景或问题 - 方案 - 取舍 - 验证或边界”展开；若被追问到具体参数、表结构或线上数据，以实际实现为准，不要临场虚构。

问题 23：离线好，为什么线上仍可能差？

AI/业务 | 中等 | 目的：评测偏差。

【参考回答】真实查询分布、文档新鲜度、权限、长尾和表述预期不同；RAG 指标偏检索而非最终答案。线上补采纳率、追问率、抽检、无答案率、P95 和反馈闭环。

【回答方向建议】先用一句结论回答，再按“背景或问题 - 方案 - 取舍 - 验证或边界”展开；若被追问到具体参数、表结构或线上数据，以实际实现为准，不要临场虚构。

问题 24：如何控制幻觉？

AI/安全 | 高难 | 目的：可信 AI。

【参考回答】【项目事实】有证据评估、强制知识库、来源引用。配合无证据拒答、答案限定上下文、引用校验和高风险人审；不能保证零幻觉。【建议补充】prompt、阈值和自动校验。

【回答方向建议】先用一句结论回答，再按“背景或问题 - 方案 - 取舍 - 验证或边界”展开；若被追问到具体参数、表结构或线上数据，以实际实现为准，不要临场虚构。

问题 25：金融资料安全怎样设计？

安全 | 高难 | 目的：生产意识。

【参考回答】材料没有 ACL/脱敏/出域事实，不能虚构。生产化应在入库和检索双层执行权限，日志/OCR/Prompt 脱敏，工具只读与路径白名单并审计，确认模型部署边界。

【回答方向建议】先用一句结论回答，再按“背景或问题 - 方案 - 取舍 - 验证或边界”展开；若被追问到具体参数、表结构或线上数据，以实际实现为准，不要临场虚构。

问题 26: PostgreSQL 中状态怎么设计？

数据库 | 高难 | 目的：建模与并发。

【参考回答】【项目事实】会话、摘要和 Agent 状态。应区分会话元数据、消息/摘要、run/checkpoint，用 thread_id/run_id/序列号和索引，更新考虑版本/乐观锁。【建议补充】真实表、事务、retention。

【回答方向建议】先用一句结论回答，再按“背景或问题 - 方案 - 取舍 - 验证或边界”展开；若被追问到具体参数、表结构或线上数据，以实际实现为准，不要临场虚构。

问题 27: 失败尝试或复盘怎么讲？

复盘 | 中等 | 目的：自省。

【参考回答】不要编线上事故。可基于真实迭代讲 Dense 对精确实体不足后引入 BM25/RRF/Reranker；必须有真实样例和基线，否则只说是观察驱动、待补正式对比。

【回答方向建议】先用一句结论回答，再按“背景或问题 - 方案 - 取舍 - 验证或边界”展开；若被追问到具体参数、表结构或线上数据，以实际实现为准，不要临场虚构。

问题 28: 重做优先改什么？

架构/复盘 | 中等 | 目的：优先级判断。

【参考回答】优先补可验证的评测基线、端到端可观测/压测、金融资料权限安全；其次是表格、多模态、引用校验和 Diff 测试。这是建议，不是已实现功能。

【回答方向建议】先用一句结论回答，再按“背景或问题 - 方案 - 取舍 - 验证或边界”展开；若被追问到具体参数、表结构或线上数据，以实际实现为准，不要临场虚构。

第4部分：追问树：最重要的 15 个问题

主问题：为什么 Hybrid RAG？

追问 1: Dense/BM25 各解决什么？【考察】检索边界。【答】Dense 偏语义，BM25

偏字面实体；项目针对错误码/路径/配置互补。【知识】Embedding、BM25。【项目】真实样例。

追问 2: RRF 怎么算？【考察】融合细节。【答】按排名融合，避免异构分数不可比。【知识】RRF 公式。【项目】k、去重、top-k。

追问 3: 为什么需要

Reranker？【考察】召回-精排。【答】小候选集深度排序，代价是延迟。【项目】模型、候选数、P95。

主问题：指标为什么可信？

追问 1: 180 条如何采样？【考察】代表性。【答】已知五类场景；需补来源、比例和独立切分。【知识】抽样/泄漏。【项目】标注表。

追问 2: Hit Rate 的 k？【考察】口径。【答】以脚本为准，必须明确。【项目】评测配置。

追问 3: 有无基线？【考察】因果。【答】准备 Dense-only/BM25-only/无精排消融。

主问题：Agentic RAG 如何防循环？

追问 1: 证据由谁判？【考察】可靠性。【答】材料仅知 LLM 评估，需确认 prompt/阈值。【知识】LLM-as-judge。

追问 2: 何时改写？【答】证据不足才改写、记录次数。【项目】max retry。

追问 3: 仍无证据？【答】停止并返回证据不足。

主问题：Bugfix Agent 能做到什么？

追问 1: 堆栈如何定位？【答】提取路径/行号/函数/异常，筛业务帧。【项目】帧过滤规则。

追问 2: 为何 Replan？【答】工具观察可能推翻初始假设。【知识】Plan-Execute-Replan。

追问 3: Diff 能执行？【答】当前只建议，必须人审。

主问题：Checkpoint 如何恢复？

追问 1: 保存什么？【答】会话记忆、摘要、执行状态；schema 待确认。

追问 2: 工具中断？【答】需要 run/step 幂等标识，依实现确认。

追问 3: 状态升级？【答】需要版本化/迁移。

主问题：引用可靠吗？

追问 1: 元数据有哪些？【答】应有文档 ID、位置、版本；实际待确认。

追问 2: 文档更新？【答】需要版本与重建索引策略。

追问 3: 引用是否支持 claim？【答】引用降低风险，不保证逐 claim 正确，需校验。

主问题：OCR/分块如何影响检索？

追问 1: OCR 错字？【答】质量检查、失败可见，不能凭空承诺纠错。

追问 2: chunk 多大？【答】按内容和评测权衡，具体参数待确认。

追问 3: 表格怎么办？【答】需确认解析策略，机械文本化会丢结构。

主问题：模型幻觉如何控制？

追问 1: 强制 KB？【答】以证据约束，无证据拒答。

追问 2: 引用错配？【答】要 claim/citation 校验，现状待确认。

追问 3: temperature=0？【答】不够，证据、拒答和验证更核心。

主问题：性能如何设计？

追问 1: 瓶颈在哪? 【答】用 trace 判断, 常见是 LLM/Reranker/OCR/向量库。

追问 2: 10 倍先改什么? 【答】先测量, 再扩无状态 API、分离异步入库。

追问 3: 成本? 【答】记录 token、限制候选、缓存; 实际策略待确认。

主问题: SSE 如何稳定?

追问 1: 断连? 【答】run id/事件号可支持恢复, 是否实现待确认。

追问 2: 取消? 【答】应停止下游任务释放资源。

追问 3: 为何不是 WebSocket? 【答】单向服务端推送时 SSE 更简单。

主问题: 金融数据如何保护?

追问 1: 文档 ACL? 【答】入库、检索双层控制是建议, 现状待确认。

追问 2: 日志脱敏? 【答】必须确认规则与出域。

追问 3: 代码工具越权? 【答】只读、路径白名单、审计。

主问题: 为什么 Milvus?

追问 1: 索引类型? 【答】以真实 collection 配置答。【知识】HNSW/IVF。

追问 2: 过滤? 【答】需标量过滤与权限; 实现待确认。

追问 3: 何时 ES? 【答】关键词/混合/既有生态可能更适合。

主问题: 数据库状态一致性?

追问 1: 并发会话? 【答】唯一 run/thread 与乐观锁或串行化; 实际待确认。

追问 2: 摘要更新? 【答】原文/摘要分层, 关键事实保留。

追问 3: 清理? 【答】要 retention/归档策略。

主问题: 如何测试?

追问 1: 检索测试? 【答】180 条与三指标, 补口径。

追问 2: Agent 测试? 【答】工具桩、分支和退出条件回归。

追问 3: 线上验证? 【答】材料不足, 需采纳/延迟/抽检指标。

主问题: 你的贡献边界?

追问 1: 哪个文件你写? 【答】用 commit/PR 佐证。

追问 2: 为何由你做? 【答】讲问题、决策、验收。

追问 3: 同事做什么? 【答】如实说明分工。

第5部分：最终面试准备清单

1. 最值得强调的 3-5 个亮点

- ☐ Hybrid 检索有明确业务针对性：错误码、日志路径、配置项。
- ☐ Agentic RAG 具有证据评估与重检闭环。
- ☐ Bugfix 将堆栈、代码上下文、工具调用与建议 Diff 连成闭环。
- ☐ PG + Checkpointer 支持状态恢复。
- ☐ 180 条离线集上的三项检索指标可量化。

2. 最危险、最容易暴露短板的 20 个问题

- ☐ 指标定义/top-k/样本/基线；☐ 个人代码与分工；☐ BM25 参数、RRF k、Reranker 候选数；☐ 模型、Embedding、Milvus schema/index；☐ chunk/OCR/引用版本；☐ 证据评估 prompt/max retry/无答案；☐ 堆栈过滤/Diff 验证；☐ checkpoint schema/幂等恢复；☐ SSE 断连；☐ ACL/脱敏/出域；☐ QPS/P95/成本；☐ 线上收益/事故。

3. 面试前必须记住的数据

已确认：项目周期 2026.03-2026.07；评测集 180 条；场景：错误码、配置查询、日志路径、故障排查、SOP；Context Recall 0.988；Hit Rate 0.988；MRR 0.935。

必须确认：每指标口径/top-k、样本比例与标注人、基线、文档/chunk 数、P95/QPS/token 成本、用户和采纳率。

4. 面试前必须复习的技术知识

RAG：chunk/metadata、Embedding、BM25、Dense、RRF、Reranker、MRR/Recall/Hit Rate、RAGAS 边界。

Agent：ReAct、Plan-Execute-Replan、LangGraph 状态图/checkpoint、工具调用、循环退出、人审。

后端：FastAPI、SSE、超时/重试/幂等/限流、PostgreSQL 索引事务。

系统：向量索引、扩容、权限、敏感数据；AI：grounding、拒答、引用校验、离线-线上偏差。

5. 必须向项目成员确认的事实

模型/Embedding/Reranker 版本与部署；Milvus schema/index/数据量；BM25 实现；chunk/OCR 参数；LangGraph 节点、状态、重试；PG 表与恢复语义；ACL/脱敏/审计；部署监控/发布；分工和 PR；压测、线上效果及真实事故。

6. 可以诚实说明但不能编造的边界

离线检索指标不等于线上满意度；建议 Diff 不等于自动修复；已确认范围是 Python

堆栈；SSE/容错/恢复的细节需以代码确认；安全权限材料没有，必须说待确认或生产化优先项。

7. 简历项目描述优化建议

将“完成”改为“负责 X 模块，交付 Y 接口/状态节点/评测脚本”；指标加口径；没有基线不写“显著提升”；将 Diff 写成“供人工审核”。

推荐版本：负责 DevPilot 的 Hybrid RAG 与 Agent workflow，覆盖文档解析/OCR、分块、Embedding、Milvus 检索、BM25+RRF+Reranker 及引用回答；构建 180 条离线集，Hybrid RAG 达到 Context Recall 0.988、Hit Rate 0.988、MRR 0.935（指标口径在面试展开）；基于 LangGraph 实现证据评估/改写重检和基于 Python 堆栈的 Bugfix 建议流程。

8. 项目介绍推荐表达

“我把效果与边界分开说：已验证的是 180

条离线检索集三项指标；线上收益、性能和安全若没有项目数据，我不报虚数，而说明验证方法。”

“Hybrid 是因为业务有精确实体；LangGraph 是为了条件分支、重试和可恢复状态。”

“Bugfix 的价值是辅助定位和分析，建议 Diff 保留给研发审核。”

9. 准备优先级

高：指标口径/基线/样本、个人代码和分工、Hybrid/RRF/Reranker、Agent 状态与堆栈、安全与线上数据边界。

中：Milvus 索引、chunk/OCR、SSE、PG schema、容错/可观测。

低：项目未使用的 Redis/消息队列八股，不要抢在真实事实前背诵。

最后提醒：最有说服力的项目回答是“真实交付 + 清晰边界 +

可验证证据”。未确认信息不要猜；说明已知事实、风险和验证计划，反而体现工程判断。